

# The Relational Data Model and Relational Database Constraints



# Chapter Outline

- Relational Model Concepts
- EER model to relational model mappings
- Relational Model Constraints and Relational Database Schemas
- Update Operations and Dealing with Constraint Violations

# Relational Model Concepts

- The relational Model of Data is based on the concept of a Relation.
- The model was first proposed by Dr. T.F. Codd of IBM in 1970 in the following paper: "A Relational Model for Large Shared Data Banks," Communications of the ACM, June 1970.
- Consists of three componets
  - data structure- relations
  - data manipulations- relational algebra
  - data integrity- various constraints

# Informal Definitions

- **RELATION:** A table of values
  - A relation may be thought of as a **set of rows**.
  - Each row represents a fact that corresponds to a real-world **entity** or **relationship**.
  - Each row has a value of an item or set of items that uniquely identifies that row in the table.
  - Each column typically is called by its column name or column header or attribute name.

# Informal Definitions

- Key of a Relation:
  - Each row has a value of a data item (or set of items) that uniquely identifies that row in the table
    - Called the *key*
  - In the STUDENT table, SSN is the key

# Example - Figure 5.1

The diagram illustrates a database relation table. The table has a header row with columns: STUDENT, Name, SSN, HomePhone, Address, OfficePhone, Age, and GPA. Below the header are five data rows representing tuples. Annotations with arrows point to various parts of the table: 'Relation name' points to the 'STUDENT' header; 'Attributes' points to the 'Name', 'SSN', 'HomePhone', 'Address', 'OfficePhone', 'Age', and 'GPA' headers; and 'Tuples' points to the five data rows.

| STUDENT | Name            | SSN         | HomePhone | Address              | OfficePhone | Age | GPA  |
|---------|-----------------|-------------|-----------|----------------------|-------------|-----|------|
|         | Benjamin Bayer  | 305-61-2435 | 373-1616  | 2918 Bluebonnet Lane | null        | 19  | 3.21 |
|         | Katherine Ashly | 381-62-1245 | 375-4409  | 125 Kirby Road       | null        | 18  | 2.89 |
|         | Dick Davidson   | 422-11-2320 | null      | 3452 Elgin Road      | 749-1253    | 25  | 3.53 |
|         | Charles Cooper  | 489-22-1100 | 376-9821  | 265 Lark Lane        | 749-6492    | 28  | 3.93 |
|         | Barbara Benson  | 533-69-1238 | 839-8461  | 7384 Fontana Lane    | null        | 19  | 3.25 |

# Formal definitions

- The **Schema** (or description) of a Relation:
  - Denoted by  $R(A_1, A_2, \dots, A_n)$
  - $R$  is the **name** of the relation
  - The **attributes** of the relation are  $A_1, A_2, \dots, A_n$

- Example:

CUSTOMER (Cust-id, Cust-name, Address, Phone#)

- CUSTOMER is the relation name
  - Defined over the four attributes: Cust-id, Cust-name, Address, Phone#
- Each attribute has a **domain** or a set of valid values.
  - For example, the domain of Cust-id is 6 digit numbers.

# Formal definitions

- A row is called a **tuple**, which is an ordered set of values
- A column header is called an **attribute**
  - Each attribute value is derived from an appropriate domain.
- The table is called a **relation**.
  - A relation can be regarded as a **set of tuples** (rows).
- The data type describing the types of values an attribute can have is represented by a **domain** of possible values.
- Each row in the CUSTOMER table is a 4-tuple and consists of four values, for example.  
**<632895, "John Smith", "101 Main St. Atlanta, GA 30332", "(404) 894-2000">**
- A relation is a set of such tuples (rows).



# Formal Definitions

- The **degree** of a relation is the number of attributes  $n$  of its relation schema.
- **A relation schema  $R$  of degree  $n$  is denoted by**

$$R(A_1, A_2, \dots, A_n)$$

- The ***domain*** of  $A_i$  is denoted by ***dom(A<sub>i</sub>)***.

# DEFINITION SUMMARY

## Informal Terms

Table

Column

Row

Values in a column

Table Definition

Populated Table

## Formal Terms

Relation

Attribute/Domain

Tuple

Domain

Schema of a  
Relation(intension  
)

Extension

# Characteristics of Relations

- **Ordering of tuples in a relation  $r(R)$ :**
  - The tuples are *not* considered to be ordered, even though they appear to be in the tabular form.
- **Ordering of attributes in a relation schema  $R$  (and of values within each tuple):**
  - We will consider the attributes in  $R(A_1, A_2, \dots, A_n)$  and the values in  $t = \langle v_1, v_2, \dots, v_n \rangle$  to be *ordered*.
- **Values in a tuple:** All values are considered *atomic* (indivisible). A special **null** value is used to represent values that are unknown or inapplicable to certain tuples.

# CHARACTERISTICS OF RELATIONS- Figure 5.2

| STUDENT | Name            | SSN         | HomePhone | Address              | OfficePhone | Age | GPA  |
|---------|-----------------|-------------|-----------|----------------------|-------------|-----|------|
|         | Dick Davidson   | 422-11-2320 | null      | 3452 Elgin Road      | 749-1253    | 25  | 3.53 |
|         | Barbara Benson  | 533-69-1238 | 839-8461  | 7384 Fontana Lane    | null        | 19  | 3.25 |
|         | Charles Cooper  | 489-22-1100 | 376-9821  | 265 Lark Lane        | 749-6492    | 28  | 3.93 |
|         | Katherine Ashly | 381-62-1245 | 375-4409  | 125 Kirby Road       | null        | 18  | 2.89 |
|         | Benjamin Bayer  | 305-61-2435 | 373-1616  | 2918 Bluebonnet Lane | null        | 19  | 3.21 |

# Data Integrity : Relational Integrity Constraints

- Constraints are **conditions** that must hold on **all** valid relation states.
- Static and dynamic integrity constraints
- Dynamic constraints: define side effects of various transactions(insert,delete, update) on database  
e.g triggers
- Static constraints: There are three *main types* of static constraints that define valid states of data in the relational model:
  - **Key** constraints
  - **Entity integrity** constraints
  - **Referential integrity** constraints
- Another implicit constraint is the **domain** constraint
  - Every value in a tuple must be from the *domain of its attribute* (or it could be **null**, if allowed for that attribute)

# Key Constraints

- **Superkey** of R: Is a set of attributes SK of R with the following condition:
  - No two tuples in any valid relation state  $r(R)$  will have the same value for SK
  - That is, for any distinct tuples  $t1$  and  $t2$  in  $r(R)$ ,  $t1[SK] \neq t2[SK]$
  - This condition must hold in *any valid state*  $r(R)$
- **Key** of R:
  - A "**minimal**" superkey
  - That is, a key is a superkey K such that removal of any attribute from K results in a set of attributes that is not a superkey (does not possess the superkey **uniqueness** property)

## Consider the following table Students

| StudentID | FirstName | LastName | Email               | PhoneNumber  |
|-----------|-----------|----------|---------------------|--------------|
| 1         | Alice     | Smith    | alice@example.com   | 123-456-7890 |
| 2         | Bob       | Brown    | bob@example.com     | 234-567-8901 |
| 3         | Charlie   | Clark    | charlie@example.com | 345-678-9012 |

:

### Superkeys

A superkey is any combination of columns that can uniquely identify each row. Some examples of superkeys in this table are:

- 1.{StudentID}
- 2.{StudentID, FirstName}
- 3.{Email}
- 4.{PhoneNumber}
- 5.{FirstName, LastName, Email}
- 6.{StudentID, Email}
- 7.{Email, PhoneNumber}

### Minimal Superkeys (Candidate Keys)

A minimal superkey is a superkey from which no column can be removed without losing the unique identification property. In other words, it's the smallest possible superkey. Some examples of minimal superkeys (candidate keys) in this table are:

- 1.{StudentID}
- 2.{Email}
- 3.{PhoneNumber}

Here, {StudentID}, {Email}, and {PhoneNumber} are minimal superkeys because removing any column from these sets would mean they can no longer uniquely identify each row.

---

# Key Constraints (continued)

- Example: Consider the CAR relation schema:  
 $CAR(State, Reg\#, SerialNo, Make, Model, Year)$ 
  - CAR has two keys:
    - Key1 = {State, Reg#}
    - Key2 = {SerialNo}
  - Both are also superkeys of CAR
  - {SerialNo, Make} is a superkey but *not* a key as has extra attribute.
- In general:
  - Any *key* is a *superkey* (but not vice versa)
  - Any set of attributes that *includes a key* is a *superkey*
  - A *minimal* superkey is also a key



# Key Constraints (continued)

- If a relation has several **candidate keys**, one is chosen arbitrarily to be the **primary key**.
  - The primary key attributes are underlined.
- Example: Consider the CAR relation schema:  
*CAR*(State, Reg#, SerialNo, Make, Model, Year)  
We chose SerialNo as the primary key
- The primary key value is used to *uniquely identify* each tuple in a relation and provides the tuple identity
- Also used to *reference* the tuple from another tuple
  - General rule: Choose as primary key the smallest of the candidate keys (in terms of size)
  - Not always applicable – choice is sometimes subjective

# CAR table with two candidate keys – LicenseNumber chosen as Primary Key

CAR

| <u>License_number</u> | Engine_serial_number | Make       | Model   | Year |
|-----------------------|----------------------|------------|---------|------|
| Texas ABC-739         | A69352               | Ford       | Mustang | 02   |
| Florida TVP-347       | B43696               | Oldsmobile | Cutlass | 05   |
| New York MPO-22       | X83554               | Oldsmobile | Delta   | 01   |
| California 432-TFY    | C43742               | Mercedes   | 190-D   | 99   |
| California RSK-629    | Y82935               | Toyota     | Camry   | 04   |
| Texas RSK-629         | U028365              | Jaguar     | XJS     | 04   |

**Figure 5.4**

The CAR relation, with  
two candidate keys:  
License\_number and  
Engine\_serial\_number.

# Entity Integrity

- **Entity Integrity:**

- The *primary key attributes* PK of each relation schema R in S cannot have null values in any tuple of  $r(R)$ .
  - This is because primary key values are used to *identify* the individual tuples.
  - $t[PK] \neq \text{null}$  for any tuple  $t$  in  $r(R)$
  - If PK has several attributes, null is not allowed in any of these attributes
- Note: Other attributes of R may be constrained to disallow null values, even though they are not members of the primary key.

# Referential Integrity

- A constraint involving *two* relations.
- Used to specify a *relationship* among tuples in two relations: the **referencing relation** and the **referenced relation**.
- Tuples in the *referencing relation*  $R_1$  have attributes **FK** (called **foreign key** attributes) that reference the primary key attributes **PK** of the *referenced relation*  $R_2$ . A tuple  $t_1$  in  $R_1$  is said to **reference** a tuple  $t_2$  in  $R_2$  if  $t_1[\text{FK}] = t_2[\text{PK}]$ .
- A referential integrity constraint can be displayed in a relational database schema as a directed arc from  $R_1.FK$  to  $R_2.PK$

# **Relational Database Design by ER- and EERR-to- Relational Mapping**

# Relational Databases and Relational Database Schemas

- **Relational Database Schema:**
    - A set  $S$  of relation schemas that belong to the same database.
    - $S$  is the name of the whole **database schema**
    - $S = \{R_1, R_2, \dots, R_n\}$
    - $R_1, R_2, \dots, R_n$  are the names of the individual **relation schemas** within the database  $S$
  - Following slide shows a COMPANY database schema with 6 relation schemas
-

# Schema Diagram for the COMPANY Relational Database Schema

## EMPLOYEE

|       |       |       |            |       |         |     |        |           |     |
|-------|-------|-------|------------|-------|---------|-----|--------|-----------|-----|
| Fname | Minit | Lname | <u>Ssn</u> | Bdate | Address | Sex | Salary | Super_ssn | Dno |
|-------|-------|-------|------------|-------|---------|-----|--------|-----------|-----|

## DEPARTMENT

|       |                |         |                |
|-------|----------------|---------|----------------|
| Dname | <u>Dnumber</u> | Mgr_ssn | Mgr_start_date |
|-------|----------------|---------|----------------|

## DEPT\_LOCATIONS

|                |                  |
|----------------|------------------|
| <u>Dnumber</u> | <u>Dlocation</u> |
|----------------|------------------|

## PROJECT

|       |                |           |      |
|-------|----------------|-----------|------|
| Pname | <u>Pnumber</u> | Plocation | Dnum |
|-------|----------------|-----------|------|

## WORKS\_ON

|             |            |       |
|-------------|------------|-------|
| <u>Essn</u> | <u>Pno</u> | Hours |
|-------------|------------|-------|

## DEPENDENT

|             |                       |     |       |              |
|-------------|-----------------------|-----|-------|--------------|
| <u>Essn</u> | <u>Dependent_name</u> | Sex | Bdate | Relationship |
|-------------|-----------------------|-----|-------|--------------|

**Figure 5.5**  
Schema diagram for  
the COMPANY  
relational database  
schema.

- **ER-to-Relational Mapping Algorithm**

Step 1: Mapping of Regular Entity Types

Step 2: Mapping of Weak Entity Types

Step 3: Mapping of Binary 1:1 Relation Types

Step 4: Mapping of Binary 1:N Relationship Types.

Step 5: Mapping of Binary M:N Relationship Types.

Step 6: Mapping of Multivalued attributes.

Step 7: Mapping of N-ary Relationship Types.

- **Mapping EER Model Constructs to Relations**

Step 8: Options for Mapping Specialization or Generalization.

Step 9: Mapping of Union Types (Categories).



# ER-to-Relational Mapping Algorithm

- **Step 1: Mapping of Regular Entity Types.**
  - For each regular (strong) entity type E in the ER schema, create a relation R that includes all the simple attributes of E.
  - Choose one of the key attributes of E as the primary key for R. If the chosen key of E is composite, the set of simple attributes that form it will together form the primary key of R.

**Example:** We create the relations EMPLOYEE, DEPARTMENT, and PROJECT in the relational schema corresponding to the regular entities in the ER diagram. SSN, DNUMBER, and PNUMBER are the primary keys for the relations EMPLOYEE, DEPARTMENT, and PROJECT as shown.

# ER-to-Relational Mapping Algorithm (cont)

- **Step 2: Mapping of Weak Entity Types**

- For each weak entity type  $W$  in the ER schema with owner entity type  $E$ , create a relation  $R$  and include all simple attributes (or simple components of composite attributes) of  $W$  as attributes of  $R$ .
- In addition, include as foreign key attributes of  $R$  the primary key attribute(s) of the relation(s) that correspond to the owner entity type(s).
- The primary key of  $R$  is the *combination* of the primary key(s) of the owner(s) and the partial key of the weak entity type  $W$ , if any.

**Example:** Create the relation DEPENDENT in this step to correspond to the weak entity type DEPENDENT. Include the primary key SSN of the EMPLOYEE relation as a foreign key attribute of DEPENDENT (renamed to ESSN).

The primary key of the DEPENDENT relation is the combination {ESSN, DEPENDENT\_NAME} because DEPENDENT\_NAME is the partial key of DEPENDENT.

# ER-to-Relational Mapping Algorithm (cont)

- **Step 3: Mapping of Binary 1:1 Relation Types**

For each binary 1:1 relationship type R in the ER schema, identify the relations S and T that correspond to the entity types participating in R. There are three possible approaches:

(1)Foreign Key approach: Choose the relations with ***total participation*** in R – say S – and include T's primary key in S.

**Example:** 1:1 relation MANAGES is mapped by choosing the participating entity type DEPARTMENT to serve in the role of S, because its participation in the MANAGES relationship type is total.

(2)Merged relation option: An alternate mapping of a 1:1 relationship type is possible by merging the two entity types and the relationship into a single relation. This may be appropriate when ***both participations are total***.

(3)Cross-reference or relationship relation option: The third alternative is to set up a third relation ***W(T.primarykey, S.primaryKey)*** for the purpose of cross-referencing the primary keys of the two relations S and T representing the entity types.

## ER-to-Relational Mapping Algorithm (cont)

- **Step 4: Mapping of Binary 1:N Relationship Types.**
  - For each regular binary 1:N relationship type R, identify the relation S that represent the participating entity type at the N-side of the relationship type.
  - Include as foreign key in S the primary key of the relation T that represents the other entity type participating in R.
  - Include any simple attributes of the 1:N relation type as attributes of S.

**Example:** 1:N relationship types WORKS\_FOR, CONTROLS, and SUPERVISION in the figure. For WORKS\_FOR we include the primary key DNUMBER of the DEPARTMENT relation as foreign key in the EMPLOYEE relation and call it DNO.

# ER-to-Relational Mapping Algorithm (cont)

- **Step 5: Mapping of Binary M:N Relationship Types.**
  - For each regular binary M:N relationship type R, *create a new relation S* to represent R.
  - Include as foreign key attributes in S the primary keys of the relations that represent the participating entity types; *their combination will form the primary key* of S.
  - Also include any simple attributes of the M:N relationship type (or simple components of composite attributes) as attributes of S

**Example:** The M:N relationship type WORKS\_ON from the ER diagram is mapped by creating a relation WORKS\_ON in the relational database schema. The primary keys of the PROJECT and EMPLOYEE relations are included as foreign keys in WORKS\_ON and renamed PNO and ESSN, respectively.

Attribute HOURS in WORKS\_ON represents the HOURS attribute of the relation type. The primary key of the WORKS\_ON relation is the combination of the foreign key attributes {ESSN, PNO}.

# ER-to-Relational Mapping Algorithm (cont)

- **Step 6: Mapping of Multivalued attributes.**
  - For each multivalued attribute A, create a new relation R. This relation R will include an attribute corresponding to A, plus the primary key attribute K-as a foreign key in R-of the relation that represents the entity type that has A as an attribute.
  - The primary key of R is the combination of A and K. If the multivalued attribute is composite, we include its simple components.

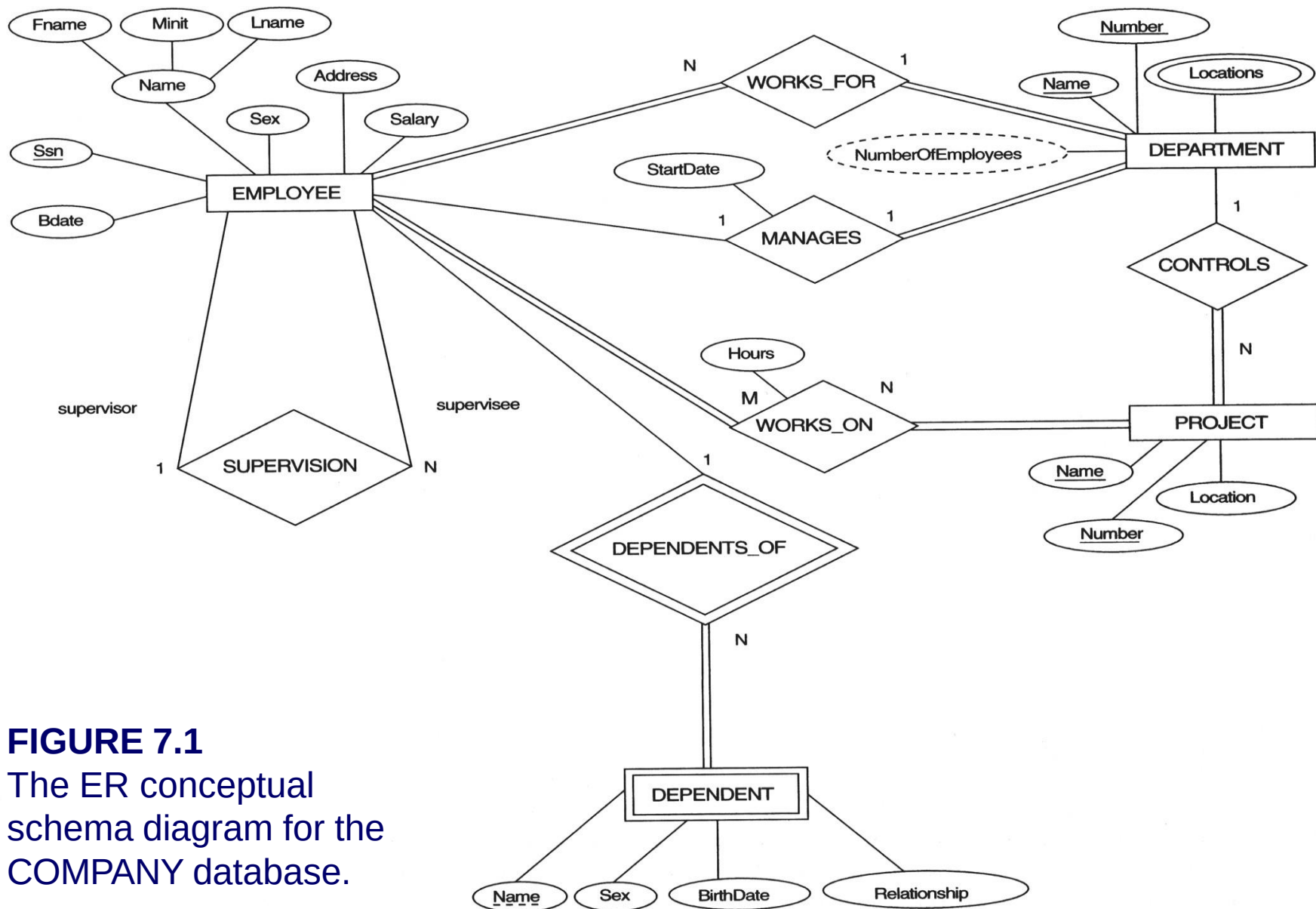
**Example:** The relation DEPT\_LOCATIONS is created. The attribute DLOCATION represents the multivalued attribute LOCATIONS of DEPARTMENT, while DNUMBER-as foreign key-represents the primary key of the DEPARTMENT relation. The primary key of R is the combination of {DNUMBER, DLOCATION}.

# ER-to-Relational Mapping Algorithm (contd.)

- **Step 7: Mapping of N-ary Relationship Types.**
  - For each n-ary relationship type R, where  $n > 2$ , create a new relationship S to represent R.
  - Include as foreign key attributes in S the primary keys of the relations that represent the participating entity types.
  - Also include any simple attributes of the n-ary relationship type (or simple components of composite attributes) as attributes of S.

**Example:** The relationship type SUPPLY in the ER on the next slide.

This can be mapped to the relation SUPPLY shown in the relational schema, whose primary key is the combination of the three foreign keys {SNAME, PARTNUM, JOBNAME}

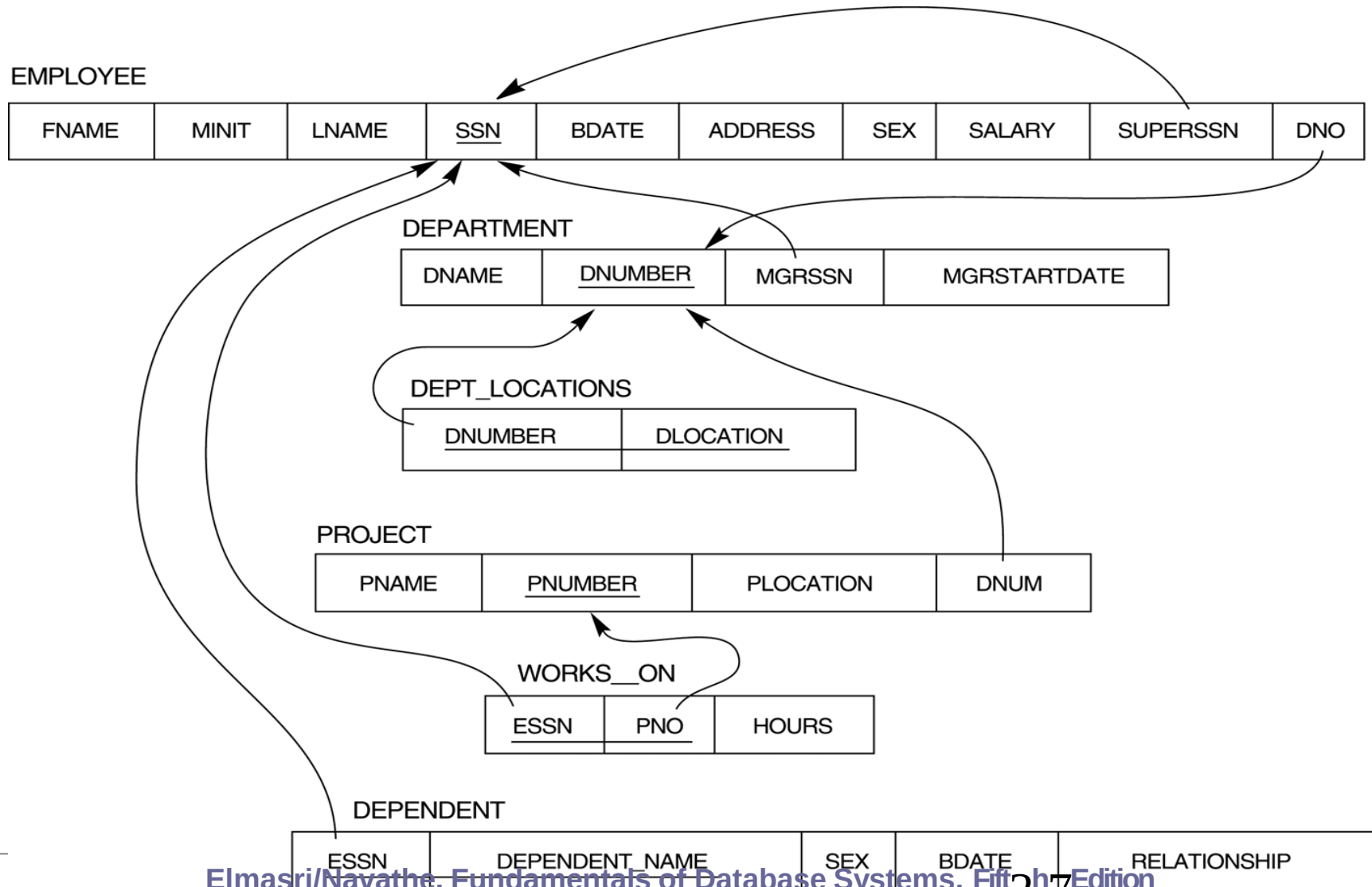


**FIGURE 7.1**  
The ER conceptual  
schema diagram for the  
COMPANY database.



## FIGURE 7.2

Result of mapping the COMPANY ER schema into a relational schema.



# Summary of Mapping constructs and constraints

***Table 7.1 Correspondence between ER and Relational Models***

## **ER Model**

Entity type

1:1 or 1:N relationship type

M:N relationship type

$n$ -ary relationship type

Simple attribute

Composite attribute

Multivalued attribute

Value set

Key attribute

## **Relational Model**

“Entity” relation

Foreign key (or “relationship” relation)

“Relationship” relation and two foreign keys

“Relationship” relation and  $n$  foreign keys

Attribute

Set of simple component attributes

Relation and foreign key

Domain

Primary (or secondary) key

# Advantages of relational model:

1. Data independence: shows clear boundry between logical and physical aspects of database management
- 2.Simplicity: simple structure so easy to use and communicate to wide variety of users in an enterprise
- 3.Set processing: facilities for manipulation of a set of records at a time
- 4.Sound theoretical background: to provide theoretical background for the database management system.

# Mapping EER Model Constructs to Relations

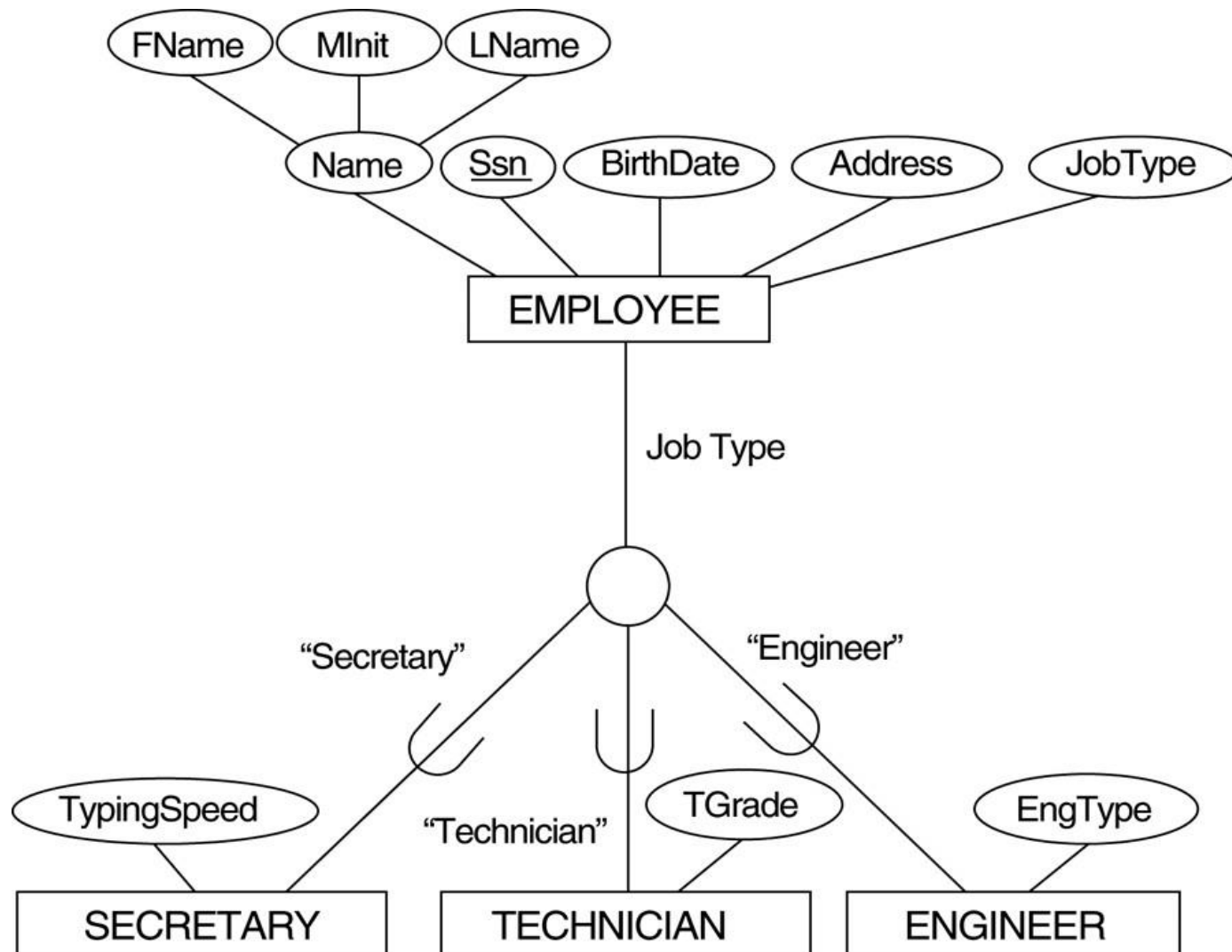
- **Step8: Options for Mapping Specialization or Generalization.**
  - Convert each specialization with  $m$  subclasses  $\{S_1, S_2, \dots, S_m\}$  and generalized superclass  $C$ , where the attributes of  $C$  are  $\{k, a_1, \dots, a_n\}$  and  $k$  is the (primary) key, into relational schemas using one of the four following options:
    - **Option 8A:** Multiple relations-Superclass and subclasses
    - **Option 8B:** Multiple relations-Subclass relations only
    - **Option 8C:** Single relation with one type attribute
    - **Option 8D:** Single relation with multiple type attributes

# Mapping EER Model Constructs to Relations

- **Option 8A: Multiple relations-Superclass and subclasses**
  - Create a relation  $L$  for  $C$  as  $L\{k, a_1, \dots, a_n\}$ , and  $PK(L) = k$ . Create a relation  $L_i$  for each subclass  $S_i$ ,  $1 < i < m$ ,  $L_i\{k, s_{i1}, \dots, s_{iik}\}$ , where  $s_{i1}, \dots, s_{iik}$  are the local attributes of  $S_i$ , and  $PK(L_i) = k$ . This option works for **any specialization** (total or partial, disjoint or overlapping).
- **Option 8B: Multiple relations-Subclass relations only**
  - Create a relation  $L_i$  for each subclass  $S_i$ ,  $1 < i < m$ , with the attributes  $Attr(L_i) = \{\text{attributes of } S_i\} \cup \{k, a_1, \dots, a_n\}$  and  $PK(L_i) = k$ . This option **only** works for a specialization whose subclasses are **total** (every entity in the superclass must belong to (at least) one of the subclasses).

**FIGURE 4.4**

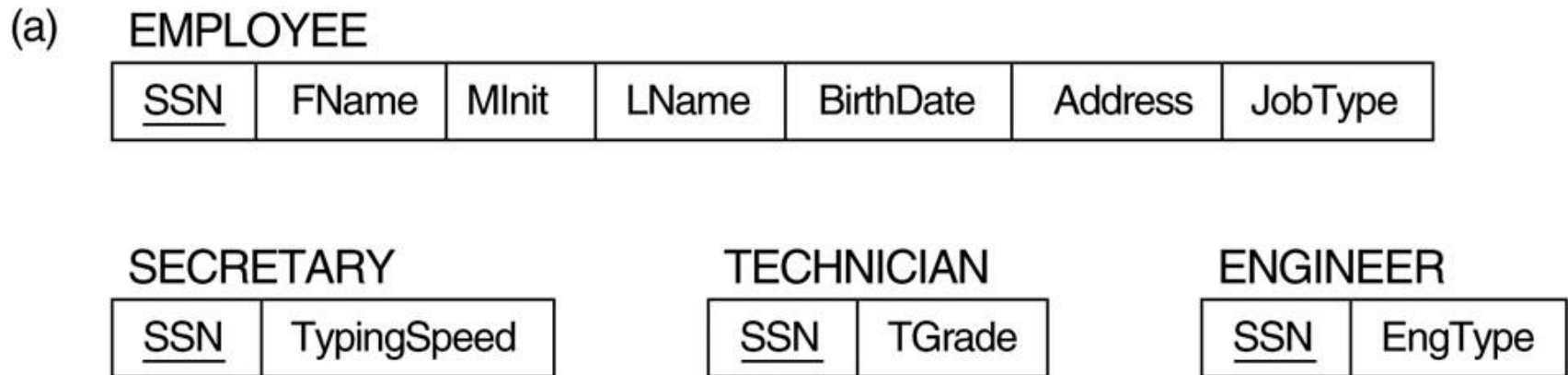
EER diagram notation for an attribute-defined specialization on JobType.



## FIGURE 7.4

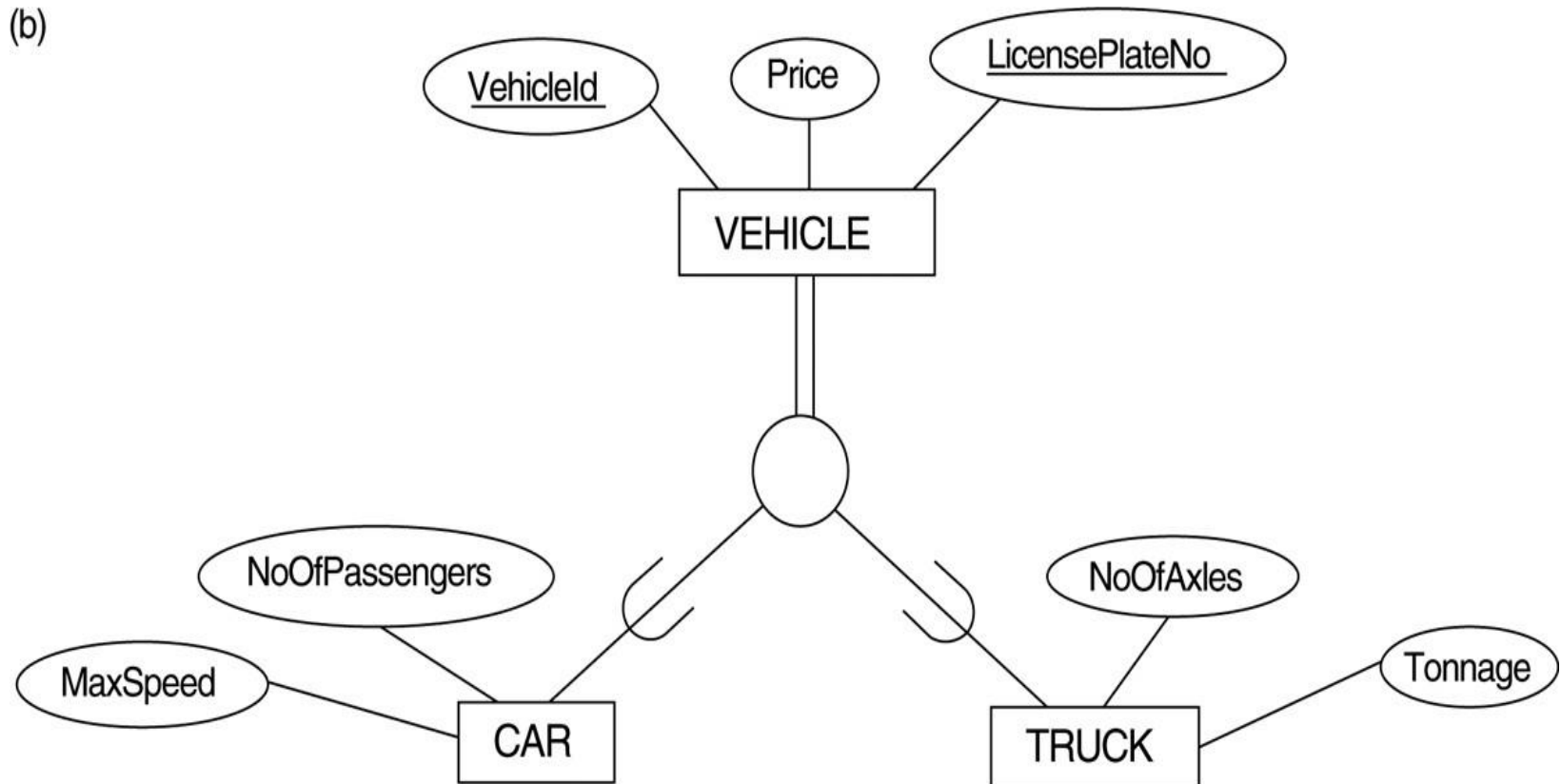
Options for mapping specialization or generalization.

(a) Mapping the EER schema in Figure 4.4 using option 8A.



## FIGURE 4.3

Generalization. (b) Generalizing CAR and TRUCK into the superclass VEHICLE.





## FIGURE 7.4

Options for mapping specialization or generalization.

(b) Mapping the EER schema in Figure 4.3b using option 8B.

(b) CAR

|                  |                |       |          |                |
|------------------|----------------|-------|----------|----------------|
| <u>VehicleId</u> | LicensePlateNo | Price | MaxSpeed | NoOfPassengers |
|------------------|----------------|-------|----------|----------------|

TRUCK

|                  |                |       |           |  |
|------------------|----------------|-------|-----------|--|
| <u>VehicleId</u> | LicensePlateNo | Price | NoOfAxles |  |
|------------------|----------------|-------|-----------|--|

# Mapping EER Model Constructs to Relations (cont)

## Option 8C: Single relation with one type attribute.

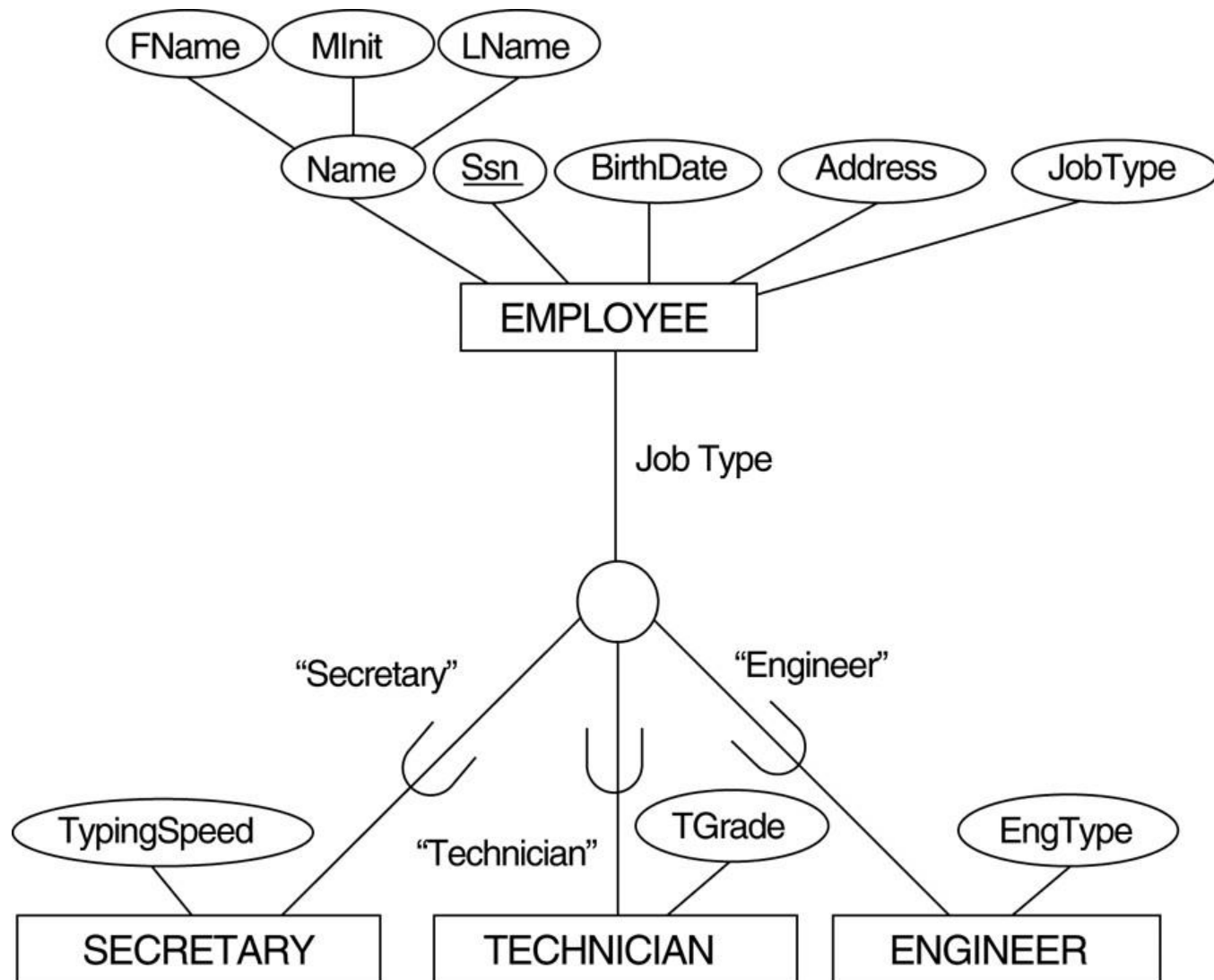
Create a single relation  $L$  with attributes  $\text{Attrs}(L) = \{k, a_1, \dots, a_n\} \cup \{\text{attributes of } S_1\} \cup \dots \cup \{\text{attributes of } S_m\} \cup \{t\}$  and  $\text{PK}(L) = k$ . The attribute  $t$  is called a type (or **discriminating**) attribute that indicates the subclass to which each tuple belongs

## Option 8D: Single relation with multiple type attributes.

Create a single relation schema  $L$  with attributes  $\text{Attrs}(L) = \{k, a_1, \dots, a_n\} \cup \{\text{attributes of } S_1\} \cup \dots \cup \{\text{attributes of } S_m\} \cup \{t_1, t_2, \dots, t_m\}$  and  $\text{PK}(L) = k$ . Each  $t_i$ ,  $1 < i < m$ , is a Boolean type attribute indicating whether a tuple belongs to the subclass  $S_i$ .

**FIGURE 4.4**

EER diagram notation for an attribute-defined specialization on JobType.



## FIGURE 7.4

Options for mapping specialization or generalization.

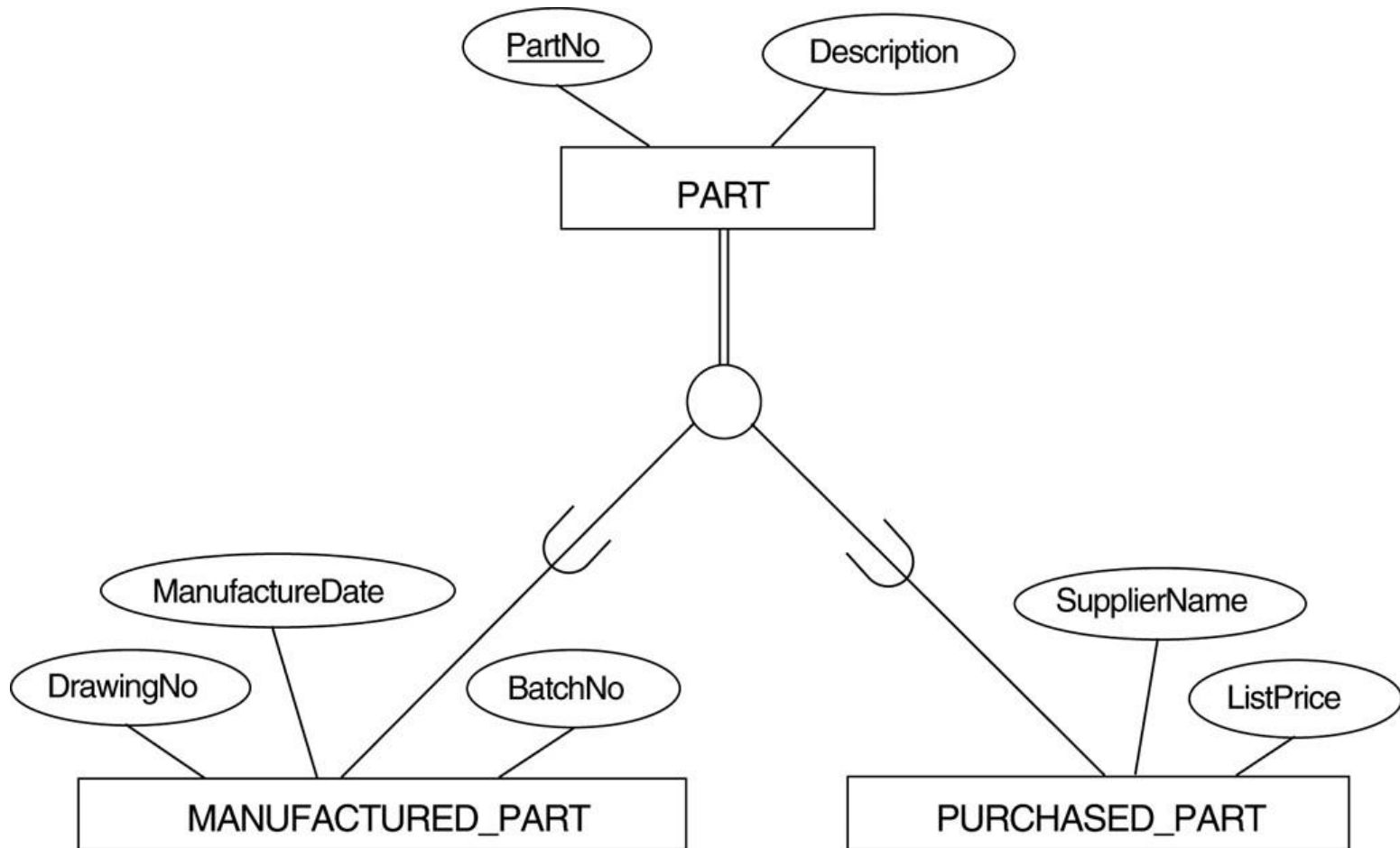
(c) Mapping the EER schema in Figure 4.4 using option 8C.

(c) EMPLOYEE

|            |       |       |       |           |         |         |             |        |  |
|------------|-------|-------|-------|-----------|---------|---------|-------------|--------|--|
| <u>SSN</u> | FName | MInit | LName | BirthDate | Address | JobType | TypingSpeed | TGrade |  |
|------------|-------|-------|-------|-----------|---------|---------|-------------|--------|--|

**FIGURE 4.5**

EER diagram notation for an overlapping (nondisjoint) specialization.



## FIGURE 7.4

Options for mapping specialization or generalization.

(d) Mapping Figure 4.5 using option 8D with Boolean type fields Mflag and Pflag.

(d) PART

| <u>PartNo</u> | Description | MFlag | DrawingNo | ManufactureDate | BatchNo | PFlag | SupplierName | ListPrice |
|---------------|-------------|-------|-----------|-----------------|---------|-------|--------------|-----------|
|---------------|-------------|-------|-----------|-----------------|---------|-------|--------------|-----------|

# Mapping EER Model Constructs to Relations (cont)

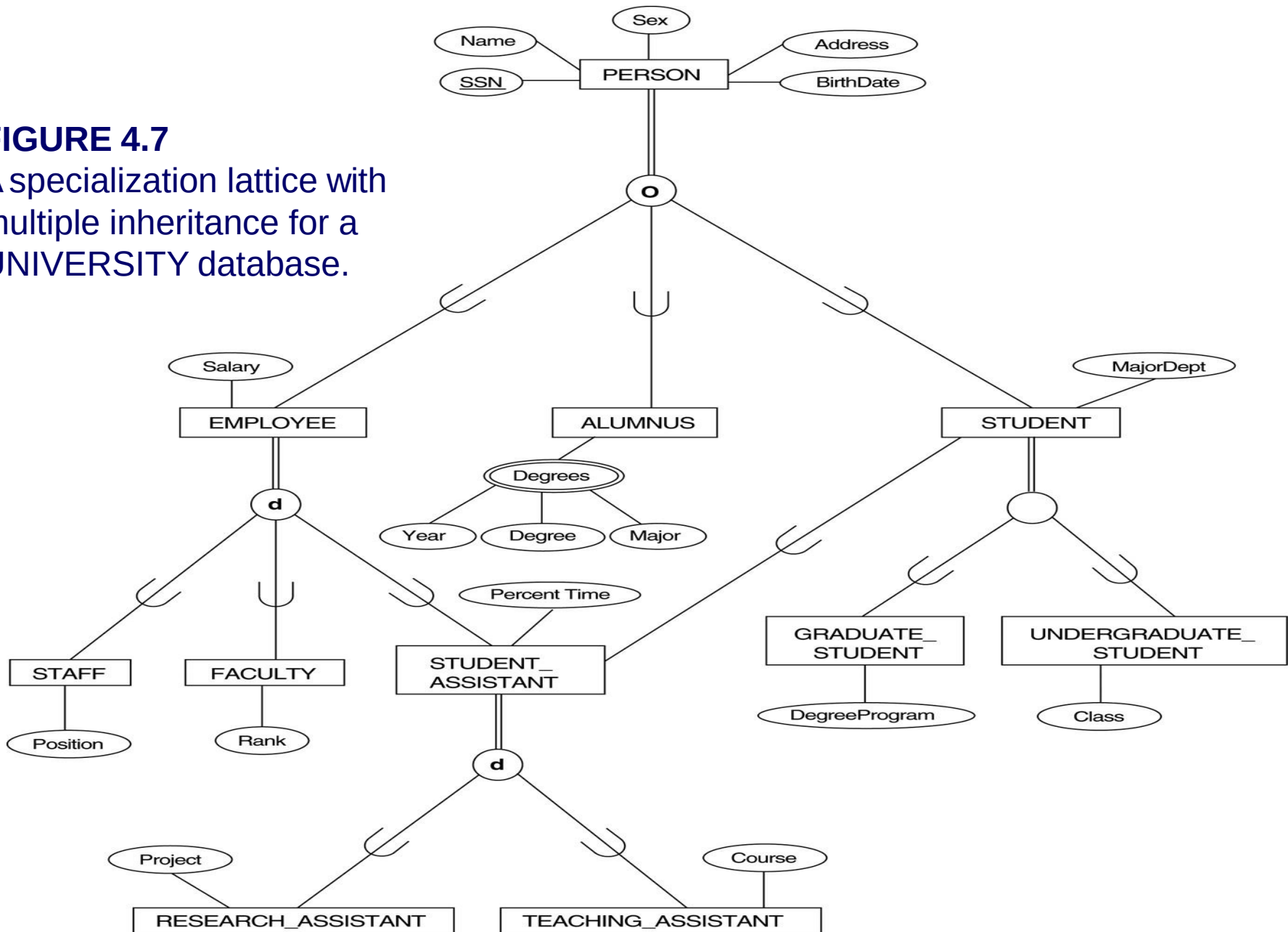
- **Mapping of Shared Subclasses (Multiple Inheritance)**

A shared subclass, such as STUDENT\_ASSISTANT, is a subclass of several classes, indicating multiple inheritance. These classes must all have the same key attribute; otherwise, the shared subclass would be modeled as a category.

We can apply any of the options discussed in Step 8 to a shared subclass, subject to the restriction discussed in Step 8 of the mapping algorithm. Below both 8C and 8D are used for the shared class STUDENT\_ASSISTANT.

**FIGURE 4.7**

A specialization lattice with multiple inheritance for a UNIVERSITY database.





## FIGURE 7.5

Mapping the EER specialization lattice in Figure 4.6 using multiple options.

### PERSON

|            |      |           |     |         |
|------------|------|-----------|-----|---------|
| <u>SSN</u> | Name | BirthDate | Sex | Address |
|------------|------|-----------|-----|---------|

### EMPLOYEE

|            |        |              |          |      |             |        |        |         |  |
|------------|--------|--------------|----------|------|-------------|--------|--------|---------|--|
| <u>SSN</u> | Salary | EmployeeType | Position | Rank | PercentTime | RAFlag | TAFlag | Project |  |
|------------|--------|--------------|----------|------|-------------|--------|--------|---------|--|

### ALUMNUS

|            |
|------------|
| <u>SSN</u> |
|------------|

### ALUMNUS\_DEGREES

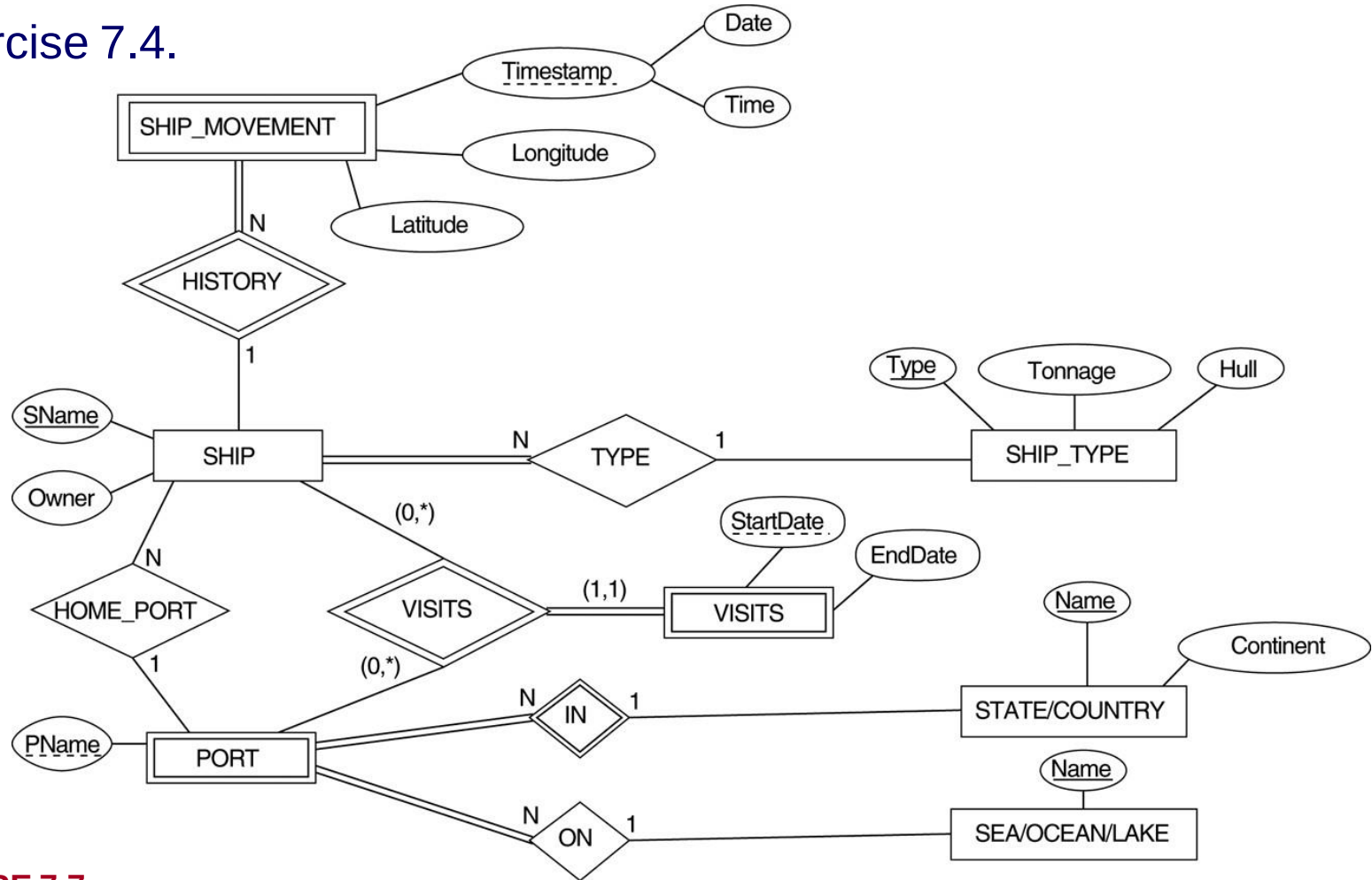
|            |      |        |  |
|------------|------|--------|--|
| <u>SSN</u> | Year | Degree |  |
|------------|------|--------|--|

### STUDENT

|            |           |          |               |               |       |                |
|------------|-----------|----------|---------------|---------------|-------|----------------|
| <u>SSN</u> | MajorDept | GradFlag | UndergradFlag | DegreeProgram | Class | StudAssistFlag |
|------------|-----------|----------|---------------|---------------|-------|----------------|

# Mapping Exercise

## Exercise 7.4.



**FIGURE 7.7**  
An ER schema for a SHIP\_TRACKING database.

# Schema Diagram for the COMPANY Relational Database Schema

## EMPLOYEE

|       |       |       |            |       |         |     |        |           |     |
|-------|-------|-------|------------|-------|---------|-----|--------|-----------|-----|
| Fname | Minit | Lname | <u>Ssn</u> | Bdate | Address | Sex | Salary | Super_ssn | Dno |
|-------|-------|-------|------------|-------|---------|-----|--------|-----------|-----|

## DEPARTMENT

|       |                |         |                |
|-------|----------------|---------|----------------|
| Dname | <u>Dnumber</u> | Mgr_ssn | Mgr_start_date |
|-------|----------------|---------|----------------|

## DEPT\_LOCATIONS

|                |                  |
|----------------|------------------|
| <u>Dnumber</u> | <u>Dlocation</u> |
|----------------|------------------|

## PROJECT

|       |                |           |      |
|-------|----------------|-----------|------|
| Pname | <u>Pnumber</u> | Plocation | Dnum |
|-------|----------------|-----------|------|

## WORKS\_ON

|             |            |       |
|-------------|------------|-------|
| <u>Essn</u> | <u>Pno</u> | Hours |
|-------------|------------|-------|

## DEPENDENT

|             |                       |     |       |              |
|-------------|-----------------------|-----|-------|--------------|
| <u>Essn</u> | <u>Dependent_name</u> | Sex | Bdate | Relationship |
|-------------|-----------------------|-----|-------|--------------|

**Figure 5.5**

Schema diagram for the COMPANY relational database schema.

# Figure 5.6 One possible database state for the COMPANY relational database schema

| EMPLOYEE | FNAME    | MINIT | LNAME   | <u>SSN</u> | BDATE      | ADDRESS                  | SEX | SALARY | SUPERSSN  | DNO |
|----------|----------|-------|---------|------------|------------|--------------------------|-----|--------|-----------|-----|
|          | John     |       | Smith   | 123456789  | 1985-01-09 | 731 Fondren, Houston, TX | M   | 30000  | 333445555 | 5   |
|          | Franklin |       | Wong    | 333445555  | 1955-12-08 | 638 Voss, Houston, TX    | M   | 40000  | 888665555 | 5   |
|          | Alicia   |       | Zelaya  | 999887777  | 1988-01-19 | 3321 Castle, Spring, TX  | F   | 25000  | 987654321 | 4   |
|          | Jennifer |       | Wallace | 987654321  | 1941-06-20 | 291 Berry, Bellaire, TX  | F   | 43000  | 888665555 | 4   |
|          | Ramesh   |       | Narayan | 866884444  | 1982-09-15 | 975 Fire Oak, Humble, TX | M   | 38000  | 333445555 | 5   |
|          | Joyce    |       | English | 453453453  | 1972-07-31 | 5631 Rice, Houston, TX   | F   | 25000  | 333445555 | 5   |
|          | Ahmad    |       | Jabbar  | 987987987  | 1989-03-29 | 980 Dallas, Houston, TX  | M   | 25000  | 987654321 | 4   |
|          | James    |       | Borg    | 888665555  | 1937-11-10 | 450 Stone, Houston, TX   | M   | 55000  | null      | 1   |

| DEPARTMENT | DNAME          | <u>DNUMBER</u> | MGRSSN    | MGRSTARTDATE | DEPT_LOCATIONS |                  |
|------------|----------------|----------------|-----------|--------------|----------------|------------------|
|            |                |                |           |              | <u>DNUMBER</u> | <u>DLOCATION</u> |
|            |                |                |           |              |                | Houston          |
|            |                |                |           |              |                | Stafford         |
|            |                |                |           |              |                | Bellaire         |
|            | Research       | 5              | 333445555 | 1988-05-22   |                | Sugarland        |
|            | Administration | 4              | 987654321 | 1995-01-01   |                |                  |
|            | Headquarters   | 1              | 888665555 | 1981-06-19   |                |                  |

| WORKS_ON | <u>ESSN</u> | <u>PNO</u> | HOURS |
|----------|-------------|------------|-------|
|          | 123456789   | 1          | 32.5  |
|          | 123456789   | 2          | 7.5   |
|          | 866884444   | 3          | 40.0  |
|          | 453453453   | 1          | 20.0  |
|          | 453453453   | 2          | 20.0  |
|          | 333445555   | 2          | 10.0  |
|          | 333445555   | 3          | 10.0  |
|          | 333445555   | 10         | 10.0  |
|          | 333445555   | 20         | 10.0  |
|          | 999887777   | 30         | 30.0  |
|          | 999887777   | 10         | 10.0  |
|          | 987987987   | 10         | 35.0  |
|          | 987987987   | 30         | 5.0   |
|          | 987654321   | 30         | 20.0  |
|          | 987654321   | 20         | 15.0  |
|          | 888665555   | 20         | null  |

| PROJECT | PNAME           | <u>PNUMBER</u> | PLOCATION | DNUM |
|---------|-----------------|----------------|-----------|------|
|         | ProductX        | 1              | Bellaire  | 5    |
|         | ProductY        | 2              | Sugarland | 5    |
|         | ProductZ        | 3              | Houston   | 5    |
|         | Computerization | 10             | Stafford  | 4    |
|         | Reorganization  | 20             | Houston   | 1    |
|         | Newbenefits     | 30             | Stafford  | 4    |

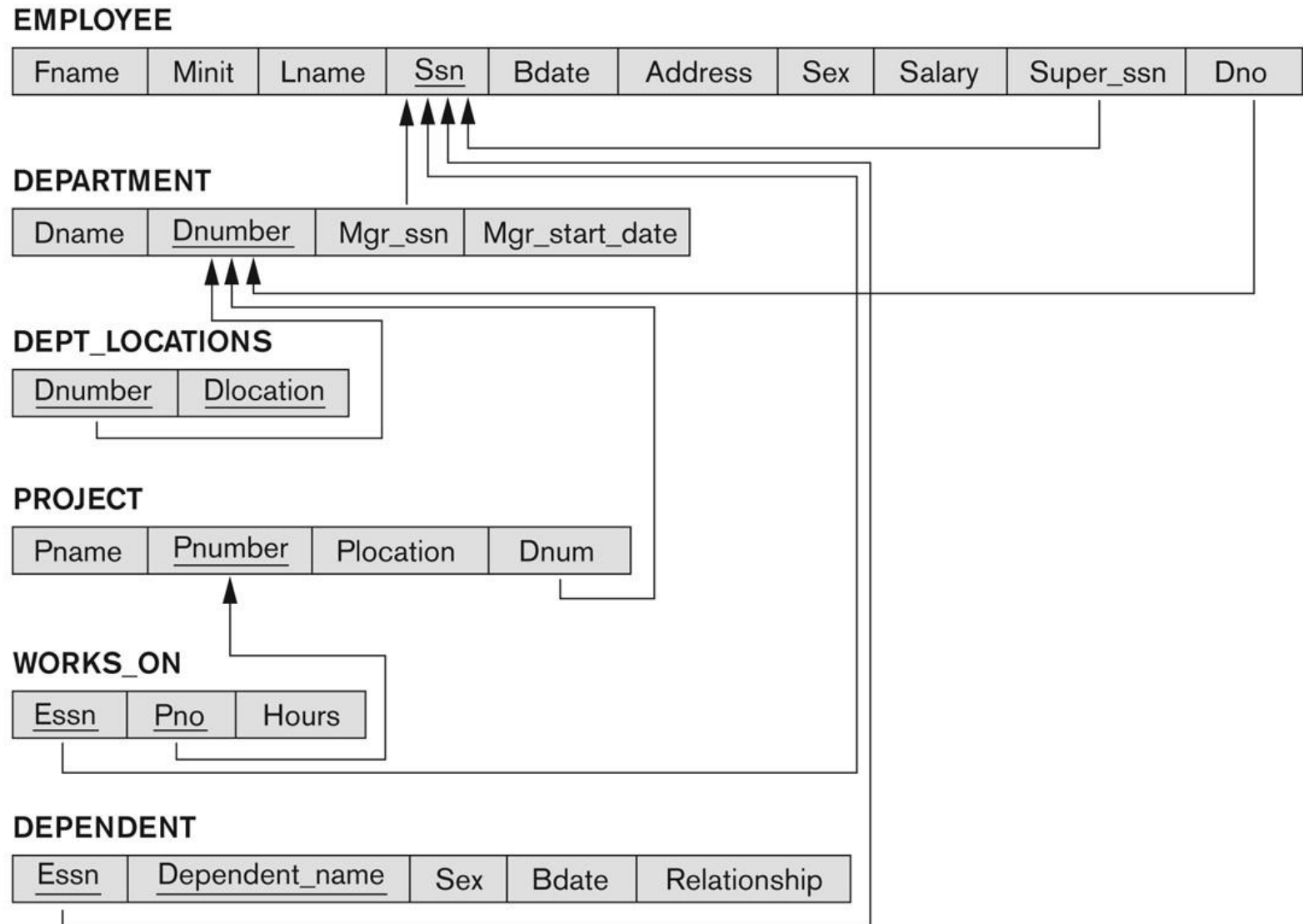
| DEPENDENT | <u>ESSN</u> | <u>DEPENDENT_NAME</u> | SEX | BDATE      | RELATIONSHIP |
|-----------|-------------|-----------------------|-----|------------|--------------|
|           | 333445555   | Alice                 | F   | 1988-04-05 | DAUGHTER     |
|           | 333445555   | Theodore              | M   | 1983-10-25 | SON          |
|           | 333445555   | Joy                   | F   | 1958-05-03 | SPOUSE       |
|           | 987654321   | Abner                 | M   | 1942-02-28 | SPOUSE       |
|           | 123456789   | Michael               | M   | 1988-01-04 | SON          |
|           | 123456789   | Alice                 | F   | 1988-12-30 | DAUGHTER     |
|           | 123456789   | Elizabeth             | F   | 1967-05-05 | SPOUSE       |

# Displaying a relational database schema and its constraints

- Each relation schema can be displayed as a row of attribute names
- The name of the relation is written above the attribute names
- The primary key attribute (or attributes) will be underlined
- A foreign key (referential integrity) constraints is displayed as a directed arc (arrow) from the foreign key attributes to the referenced table
  - Can also point the primary key of the referenced relation for clarity
- Next slide shows the COMPANY **relational schema diagram**

**Figure 5.7**

Referential integrity constraints displayed on the COMPANY relational database schema.



# Referential Integrity Constraint

- The value in the foreign key column (or columns) FK of the **referencing relation** R1 can be **either**:
  - 1) a value of an existing primary key value of a corresponding primary key PK in the **referenced relation** R2, or
  - 2) a **null**.
- In case (2), the FK in R1 should **not** be a part of its own primary key.
- Example:

<"John", "L", "Smith", 111222333, 1965-10-21, "101 Main St. Atlanta, GA 30332", M, 42000, 444555666, **NULL**>

<"Mary", "J", "Burton", 111111111, 1972-1-18, "23 Maple St. Atlanta, GA 30310", F, 35000, **NULL**, 3>

# Other Types of Constraints

- Semantic Integrity Constraints:
  - based on application semantics and cannot be expressed by the model per se
  - Example: “the max. no. of hours per employee for all projects he or she works on is 56 hrs per week”
- A **constraint specification** language may have to be used to express these
- SQL-99 allows triggers and **ASSERTIONS** to express for some of these



# Populated database state

- Each *relation* will have many tuples in its current relation state
- The **relational database state** is a union of all the individual relation states
- Whenever the database is changed, a new state arises
- Basic operations for changing the database:
  - INSERT a new tuple in a relation
  - DELETE an existing tuple from a relation
  - MODIFY an attribute of an existing tuple
- Next slide shows an example state for the COMPANY database

# Populated database state for COMPANY

**Figure 5.6**

One possible database state for the COMPANY relational database schema.

## EMPLOYEE

| Fname    | Minit | Lname   | Ssn       | Bdate      | Address                  | Sex | Salary | Super_ssn | Dno |
|----------|-------|---------|-----------|------------|--------------------------|-----|--------|-----------|-----|
| John     | B     | Smith   | 123456789 | 1965-01-09 | 731 Fondren, Houston, TX | M   | 30000  | 333445555 | 5   |
| Franklin | T     | Wong    | 333445555 | 1955-12-08 | 638 Voss, Houston, TX    | M   | 40000  | 888665555 | 5   |
| Alicia   | J     | Zelaya  | 999887777 | 1968-01-19 | 3321 Castle, Spring, TX  | F   | 25000  | 987654321 | 4   |
| Jennifer | S     | Wallace | 987654321 | 1941-06-20 | 291 Berry, Bellaire, TX  | F   | 43000  | 888665555 | 4   |
| Ramesh   | K     | Narayan | 666884444 | 1962-09-15 | 975 Fire Oak, Humble, TX | M   | 38000  | 333445555 | 5   |
| Joyce    | A     | English | 453453453 | 1972-07-31 | 5631 Rice, Houston, TX   | F   | 25000  | 333445555 | 5   |
| Ahmad    | V     | Jabbar  | 987987987 | 1969-03-29 | 980 Dallas, Houston, TX  | M   | 25000  | 987654321 | 4   |
| James    | E     | Borg    | 888665555 | 1937-11-10 | 450 Stone, Houston, TX   | M   | 55000  | NULL      | 1   |

## DEPARTMENT

| Dname          | Dnumber | Mgr_ssn   | Mgr_start_date |
|----------------|---------|-----------|----------------|
| Research       | 5       | 333445555 | 1988-05-22     |
| Administration | 4       | 987654321 | 1995-01-01     |
| Headquarters   | 1       | 888665555 | 1981-06-19     |

## DEPT\_LOCATIONS

| Dnumber | Dlocation |
|---------|-----------|
| 1       | Houston   |
| 4       | Stafford  |
| 5       | Bellaire  |
| 5       | Sugarland |
| 5       | Houston   |

## WORKS\_ON

| Essn      | Pno | Hours |
|-----------|-----|-------|
| 123456789 | 1   | 32.5  |
| 123456789 | 2   | 7.5   |
| 666884444 | 3   | 40.0  |
| 453453453 | 1   | 20.0  |
| 453453453 | 2   | 20.0  |
| 333445555 | 2   | 10.0  |
| 333445555 | 3   | 10.0  |
| 333445555 | 10  | 10.0  |
| 333445555 | 20  | 10.0  |
| 999887777 | 30  | 30.0  |
| 999887777 | 10  | 10.0  |
| 987987987 | 10  | 35.0  |
| 987987987 | 30  | 5.0   |
| 987654321 | 30  | 20.0  |
| 987654321 | 20  | 15.0  |
| 888665555 | 20  | NULL  |

## PROJECT

| Pname           | Pnumber | Plocation | Dnum |
|-----------------|---------|-----------|------|
| ProductX        | 1       | Bellaire  | 5    |
| ProductY        | 2       | Sugarland | 5    |
| ProductZ        | 3       | Houston   | 5    |
| Computerization | 10      | Stafford  | 4    |
| Reorganization  | 20      | Houston   | 1    |
| Newbenefits     | 30      | Stafford  | 4    |

## DEPENDENT

| Essn      | Dependent_name | Sex | Bdate      | Relationship |
|-----------|----------------|-----|------------|--------------|
| 333445555 | Alice          | F   | 1986-04-05 | Daughter     |
| 333445555 | Theodore       | M   | 1983-10-25 | Son          |
| 333445555 | Joy            | F   | 1958-05-03 | Spouse       |
| 987654321 | Abner          | M   | 1942-02-28 | Spouse       |
| 123456789 | Michael        | M   | 1988-01-04 | Son          |
| 123456789 | Alice          | F   | 1988-12-30 | Daughter     |
| 123456789 | Elizabeth      | F   | 1967-05-05 | Spouse       |

# Update Operations on Relations

- INSERT a tuple.
- DELETE a tuple.
- MODIFY a tuple.
- Integrity constraints should not be violated by the update operations.
- Several update operations may have to be grouped together.
- Updates may **propagate** to cause other updates automatically. This may be necessary to maintain integrity constraints.

# Update Operations on Relations

- In case of integrity violation, several actions can be taken:
  - Cancel the operation that causes the violation (RESTRICT or REJECT option)
  - Perform the operation but inform the user of the violation
  - Trigger additional updates so the violation is corrected (CASCADE option, SET NULL option)
  - Execute a user-specified error-correction routine

# Possible violations for each operation

- INSERT may violate any of the constraints:
  - Domain constraint:
    - if one of the attribute values provided for the new tuple is not of the specified attribute domain
  - Key constraint:
    - if the value of a key attribute in the new tuple already exists in another tuple in the relation
  - Referential integrity:
    - if a foreign key value in the new tuple references a primary key value that does not exist in the referenced relation
  - Entity integrity:
    - if the primary key value is null in the new tuple

# Possible violations for each operation

- DELETE may violate only referential integrity:
  - If the primary key value of the tuple being deleted is referenced from other tuples in the database
    - Can be remedied by several actions: RESTRICT, CASCADE, SET NULL
      - RESTRICT option: reject the deletion
      - CASCADE option: propagate the new primary key value into the foreign keys of the referencing tuples
      - SET NULL option: set the foreign keys of the referencing tuples to NULL
  - One of the above options must be specified during database design for each foreign key constraint

# Possible violations for each operation

- UPDATE may violate domain constraint and NOT NULL constraint on an attribute being modified
- Any of the other constraints may also be violated, depending on the attribute being updated:
  - Updating the primary key (PK):
    - Similar to a DELETE followed by an INSERT
    - Need to specify similar options to DELETE
  - Updating a foreign key (FK):
    - May violate referential integrity
  - Updating an ordinary attribute (neither PK nor FK):
    - Can only violate domain constraints

# Summary

- Presented Relational Model Concepts
  - Definitions
  - Characteristics of relations
- Discussed Relational Model Constraints and Relational Database Schemas
  - Domain constraints
  - Key constraints
  - Entity integrity
  - Referential integrity
- Described the Relational Update Operations and Dealing with Constraint Violations



# In-Class Exercise

(Taken from Exercise 5.15)

Consider the following relations for a database that keeps track of student enrollment in courses and the books adopted for each course:

STUDENT(SSN, Name, Major, Bdate)

COURSE(Course#, Cname, Dept)

ENROLL(SSN, Course#, Quarter, Grade)

BOOK\_ADOPTION(Course#, Quarter, Book\_ISBN)

TEXT(Book\_ISBN, Book\_Title, Publisher, Author)

**Draw a relational schema diagram specifying the foreign keys for this schema.**

## STUDENT

SSN

Name

Major

Bdate

## COURSE

Course #

Cname

Dept

## ENROLL

SSN

Course #

Quarter

Grade

## BOOK\_ADOPTION

Course #

Quarter

Book\_ISBN

## TEXT

Book\_ISBN

Book\_Title

Publisher

Author